EXP4

August 9, 2026

[1]:
```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

[2]:
```
diabetes = pd.read_csv('diabetes.csv')
diabetes.columns
```

[2]:
```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

[3]:
```
diabetes.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

[4]:
```
X = diabetes.drop('Outcome', axis=1) # All columns except 'Outcome' are features
y = diabetes['Outcome'] # 'Outcome' is the target variable
feature_names = X.columns # Get feature names from the X DataFrame
```

```
print(feature_names)
```

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age'],
      dtype='str')
```

[5]:
```
print(X.shape)
print(y.shape)
```

```
(768, 8)
(768,)
```

[6]:
```
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)

X_scaled = (X - mean) / std

print("Mean of each feature:")
print(mean)

print("\nStandard Deviation of each feature:")
print(std)
```

```
Mean of each feature:
Pregnancies                 3.845052
Glucose                   120.894531
BloodPressure              69.105469
SkinThickness              20.536458
Insulin                    79.799479
BMI                        31.992578
DiabetesPedigreeFunction    0.471876
Age                        33.240885
dtype: float64

Standard Deviation of each feature:
Pregnancies                 3.367384
Glucose                    31.951796
BloodPressure              19.343202
SkinThickness              15.941829
Insulin                   115.168949
BMI                         7.879026
DiabetesPedigreeFunction    0.331113
Age                        11.752573
dtype: float64
```

[7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
```

```
    random_state=42
)
```

[8]:
```
lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
print(mse_lr)
print(r2_lr)
```

```
0.17104527280850096
0.2550028117674178
```

[9]:
```
ridge = Ridge()

ridge_params = {
    'alpha': [0.01, 0.1, 1, 10, 100]
}

ridge_cv = GridSearchCV(
    ridge,
    ridge_params,
    cv=5,
    scoring='neg_mean_squared_error'
)

ridge_cv.fit(X_train, y_train)

y_pred_ridge = ridge_cv.predict(X_test)

mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)
print(mse_ridge)
print(r2_ridge)
```

```
0.17065554755497536
0.2567002817605516
```

[10]:
```
lasso = Lasso(max_iter=10000)

lasso_params = {
    'alpha': [0.001, 0.01, 0.1, 1, 10]
}

lasso_cv = GridSearchCV(
```

```
    lasso,
    lasso_params,
    cv=5,
    scoring='neg_mean_squared_error'
)

lasso_cv.fit(X_train, y_train)

y_pred_lasso = lasso_cv.predict(X_test)

mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)
print(mse_lasso)
print(r2_lasso)
```

```
0.17079703772804197
0.2560840134511947
```

[11]:
```
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Ridge Regression', 'Lasso Regression'],
    'Best Alpha': ['-',
                   ridge_cv.best_params_['alpha'],
                   lasso_cv.best_params_['alpha']],
    'MSE': [mse_lr, mse_ridge, mse_lasso],
    'R2 Score': [r2_lr, r2_ridge, r2_lasso]
})

print("\nBest Ridge Alpha:", ridge_cv.best_params_['alpha'])
print("Best Lasso Alpha:", lasso_cv.best_params_['alpha'])

print("\nPerformance Comparison")
print(results)
```

```
Best Ridge Alpha: 10
Best Lasso Alpha: 0.001

Performance Comparison
               Model Best Alpha       MSE  R2 Score
0  Linear Regression          -  0.171045  0.255003
1   Ridge Regression         10  0.170656  0.256700
2   Lasso Regression      0.001  0.170797  0.256084
```